Response to Reviewers

PoolParty: streamlined design of DNA sequence libraries in Python

Zhihan Liu, Aidan Cordero, Justin B. Kinney

We thank the Editor and Reviewers for their thoughtful critiques, to which we provide a point-by-point response below. The manuscript has been improved substantially as a result of this feedback. Additional edits have also been made throughout the manuscript to improve clarity.

Editor Comments

In general, the reviewers had positive impressions of PoolParty, and it can likely be revised to a state that will be acceptable. Please carefully address each point raised by the reviewers. It's especially important to provide benchmarking data, comparisons to existing tools, and not to overstate the benefits the tool may provide or fail to discuss limitations. Reviewer 1 recommends comparing PoolParty to 2–3 other comparable tools. However, rather than arbitrarily picking two or three other tools for comparisons, you should use a well-defined process for picking which tools, and how many, to include in comparisons. Perhaps 2–3 is adequate; however, that may be too few tools for comparisons. The comparisons should include, at a minimum, benchmarking (e.g., runtime, memory usage), program features, and similarities/differences in program outputs.

We have expanded the Introduction to discuss in more depth the limitations of competing tools that PoolParty overcomes:

> *Existing tools can help with parts of this process. General-purpose sequence toolkits (Cock et al., 2009; Pereira et al., 2015; Klie and Laub, 2023) can manipulate sequences but have no notion of a variant library as a structured object. Consequently, users must write custom scripts to handle the combinatorial logic themselves. Several other tools automate library design or sequence perturbation for particular applications. For example, VaLiAnT (Barbon et al., 2022) generates mutational scanning libraries of both coding and noncoding regions; supported mutation types include single-nucleotide substitutions, amino acid substitutions, and in-frame deletions. For MPRAs, MPRAnator (Georgakopoulos-Soares et al., 2017) provides modules for motif placement, SNP design, and negative control generation (see also Ghazi et al., 2018). For in silico sequence experiments, tangermeme (Schreiber, 2025) supports sequence perturbation and analysis of genomic deep learning models. However, each of these tools is targeted to specific applications and, as a result, no individual tool can generate the full range of library designs in common use. For example, VaLiAnT does not support regulatory sequence libraries containing random combinations of transcription factor binding sites, MPRAnator does not support codon-aware mutagenesis of protein-coding sequences, and tangermeme does not provide metadata describing the construction of each individual sequence.*

The revised Discussion now better qualifies how PoolParty compares to existing tools:

> *PoolParty complements existing library-design tools such as VaLiAnT (Barbon et al., 2022) and MPRAnator (Georgakopoulos-Soares et al., 2017) by providing a single framework for DMS libraries, MPRA libraries, and library designs that combine elements of both.…Additional comparisons between PoolParty and existing tools are provided in Supplemental Information, Figs. S3 and S4 and Tables S2 and S3.*

The revised Discussion also now explicitly discusses limitations of PoolParty:

> *PoolParty was designed to solve a specific computational problem: streamlining the design of DNA sequence libraries for a wide range of experimental applications and in silico studies. As such, it has several limitations. PoolParty operates on sequences, not genomic loci or transcript models. It can accept genomic coordinates as input to extract sequences, but does not interpret transcript annotations or keep track of those coordinates through subsequent design steps (e.g., as VaLiAnT does), which may make PoolParty less helpful for certain genome-editing applications. PoolParty is also not a sequence optimization tool; it does not design sequences that have optimized codon usage (Swainston et al., 2017), that are subject to complex synthesis constraints (Zulkower and Rosser, 2020), or that have optimal activity as assessed by machine learning models (Schreiber et al., 2025). PoolParty also does not address the problem of physically constructing DNA sequence libraries, e.g., by designing mutagenic primers (Hiraga et al., 2021). Rather, we expect that PoolParty can in many cases be productively paired with complementary tools that tackle these challenges.*

We have also added a Supplemental Information document that systematically compares PoolParty to competing tools (Tables S2 and S3, Figures S3 and S4). The approach used to select these tools is described in the Supplemental Methods section of this document:

> *A literature search identified 13 tools with substantial functional overlap with PoolParty; these are listed in Table S2. Table S3 compares the specific features and capabilities of the 7 tools most similar in purpose to PoolParty. Of these 7, we judged VaLiAnT Barbon et al. (2022), MPRAnator Georgakopoulos-Soares et al. (2017), and tangermeme Schreiber (2025) to be the closest competitors. MPRAnator is a web application that, as of this writing, is no longer accessible. Figs. S3 and S4 provide explicit comparisons of the code / inputs required to construct identical libraries using PoolParty and VaLiAnT (in Fig. S3) or PoolParty and tangermeme (in Fig. S4). We note that neither VaLiAnT nor tangermeme is readily capable of generating the three libraries used in the benchmarking study of PoolParty.*

The Supplemental Information also reports benchmarking experiments that quantify the compute and memory requirements of PoolParty on the library designs featured in the main text (Fig. S2 and Table S1). We note that these benchmarks do not compare PoolParty to other tools, for two reasons. First, none of the other tools was capable of generating libraries with the same three designs. Second, the paper does not claim that PoolParty has any execution speed or memory usage advantages over other methods; rather, the competative advantage of PoolParty is its flexibility, ease of use, and the per-sequence metadata it provides.

Reviewer 1

1. In the GB1 example, the library exceeds 540,000 sequences, yet the paper provides no runtime or memory consumption benchmarks. Readers cannot assess the tool's practical usability across different library scales. The authors should supplement benchmarking data, including runtime and peak memory usage across libraries of increasing sizes (e.g., 1K, 10K, 100K, 500K sequences), and specify the test environment configuration.

This is a good suggestion. As described above, the new Supplemental Information document (Fig. S2 and Table S1) reports benchmarking tests of runtime and peak memory usage using libraries of increasing size and matching the designs featured in Figs. 2, 3, and 4. The test environment is reported is reported in the Supplemental Methods section of this document.

> *Figure S2 and Table S1 present a benchmarking study of PoolParty using the three library designs featured in Figs. 2, 3, and 4 of the main text. Benchmarking experiments were run under Ubuntu 22.04.3 LTS in Windows Subsystem for Linux 2 on an Intel Core Ultra 9 185H workstation with 31 GB RAM allocated to WSL2, using CPython 3.12.2, PoolParty 0.1.0, and StateTracker 0.1.0. Timings are single-threaded wall-clock measurements of* `generate_library`*, reported as mean ± SD over five replicate runs. Peak memory is the maximum resident set size reported by* `getrusage`*, with each data point measured in a fresh process.*

2. The paper highlights design cards as a core feature, emphasizing their use as covariates in downstream analyses. However, it does not evaluate whether design card variables provide significant incremental predictive value beyond sequence-derived features. The authors should either supplement a comparative experiment quantifying the added information gain of design cards, or explicitly state in the Discussion that this validation is planned as future work.

   Design cards record the specific design choices used to construct each sequence. In the Fig. 4 example, we demonstrate how this facilitates surrogate modeling studies by making these design choices readily available as covariates. We do not, however, mean to claim that the information in design cards increases the predictive performance of sequence-to-function models. We have clarified the intended role of design cards in the Discussion:

   > *Design cards are thus a bookkeeping device that allows users to avoid reverse-engineering sequence architecture from raw sequence when investigating how such architecture affects the output of a sequence-to-function model.*

3. The introduction mentions VaLiAnT and MPRAnator, noting their limitations, but does not provide a systematic comparison between PoolParty and these tools. The authors should provide a table comparing PoolParty with 2–3 existing tools on core features, and demonstrate code conciseness in at least one common use case, to help readers objectively assess PoolParty's advantages.

   This is a good suggestion. Tables S2 and S3 in the new Supplemental information document provide a systematic comparison between PoolParty and other competing tools. Figs. S3 and S4 further provide explicit comparisons of the code / inputs required to construct identical libraries using PoolParty and VaLiAnT. We also note in Supplemental Methods that MPRAnator (which runs through a web interface) is no longer function, and so we were unable to carry out a similar comparison with this tool.

4. Provide benchmarking data on runtime and memory usage across libraries of varying sizes.

   This is addressed in the response to Comment 1 above.

5. Include a table comparing features with 2–3 existing tools.

   This is addressed in the response to Comment 3 above.

6. To further contextualize this research within the existing academic landscape and strengthen its theoretical support, it is recommended that the authors supplement several relevant representative references.

   [1] `https://doi.org/10.1093/nsr/nwaf495`

   [2] `https://doi.org/10.1038/s41467-026-72882-y`

   [3] `https://doi.org/10.1186/s13059-025-03606-6`

   [4] `https://doi.org/10.1016/j.patcog.2025.112078`

   Having read these papers, we were unable to identify a clear connection between these references and PoolParty, or the more general problem of sequence library design: the first paper discusses multimodal representations of small molecules for drug discovery; the second paper models single-cell transcriptomic and surface-protein measurements; the third paper discusses read mapping to pangenome graphs; and the fourth paper concerns circRNA-miRNA interaction prediction. We therefore decided not to cite these papers in the manuscript.

7. Either provide empirical analysis of design cards' incremental contribution to surrogate modeling performance, or explicitly discuss this as future work.

   This is addressed in the response to Comment 2 above.

Reviewer 2

1. Additional statistical readouts describing pool generation would be useful. For example, how many unique sequences vs duplicates were produced in each pool (i.e. out of the universe of all permutations that meet the specifications)? How unique are the sequences (min/max/average hamming distance)? How frequent are homopolymers? Etc.

   This is a good suggestion. To provide this information we have added a `.stats()` method to the Pool class. This method computes a number of summary statistics about the pool, including the size of the design space (when this is fixed), the number of unique/duplicated sequences generated, the pairwise Hamming distances between generated sequences, the frequencies of homopolymer occurrences, etc. We have added a short paragraph to the Sequence metadata subsection describing this method, as well as documentation at `https://poolparty.readthedocs.io/en/latest/pool.html#summarising-a-library-stats`.

2. The authors disclose that LLMs were used to develop, debug, and document the code. Did the authors also write a test harness, and what edge cases were nominated by the LLMs for testing vs what edge cases were nominated by the human developers for testing?

Yes, the PoolParty codebase includes over 3,000 tests that can be run using pytest. In our experience, the coding agents were quite good at testing mechanical aspects of the code (e.g., invalid inputs and boundary conditions). However, human-written use cases (from which most of the manual tests were derived) were essential for testing the types of inputs and requests likely to be encountered in real biological applications. The current suite consists of a mixture of these two types of tests. We did not track the origin of each test, however, and so cannot precisely quantify how many came from each source.

3. The backward/forward pass steps are not well described in the manuscript, and figure 1b is hard to interpret. I suggest adding a supplemental figure showing how these passes are performed. For example, the text explains that the backward pass happens as information is passed to the root node – does this mean that the state is passed from 'Mutagenize' to 'Pool A' in or first from 'Pool Final' to 'Stack'? And if so, what information is passed? Perhaps showing each step in an expanded figure or labeling arrows with the order would be more useful.

   The new Supplemental Information document includes a figure, Fig. S1, that breaks down the forward and backward pass computations in much more detail. We believe this will answer the Reviewer's questions.

4. A comparison to the pools produced by other tools such as VaLiAnT or MPRAnator would provide additional confidence in the ability of PoolParty to produce sequence pools as expected. Metrics such as the number of pool elements that overlap and an investigation of any unique pool elements would ensure that the generate pools are accurate.

   Figs. S3 and S4 in the new Supplemental Information document compare the code/files needed to create identical libraries with PoolParty and VaLiAnT (Fig. S3) or tangermeme (Fig. S4). We attempted to make a similar figure for MPRAnator, but that tool uses a web interface that is currently inoperative.

5. Additional operations to improve library content would be useful. E.g. to maximize hamming distance, remove sequences containing restriction enzyme sites, remove homopolymers, etc. would make the tool more useful in generating ready-to-use pools.

   PoolParty does include a `filter` Operation that allows users to remove sequences that do not satisfy custom criteria. This Operation includes a number of built-in filter criteria, including GC content, homopolymer content, sequence complexity, and restriction enzyme recognition sites. We have also expanded the documentation on this capability and added a short description of this capability to the manuscript.

   > *Sequences can be filtered based on sequence length, GC content, homopolymer length, sequence complexity, restriction enzyme recognition sites, or other user-defined criteria.*

   We have also clarified PoolParty's limitations in the Discussion section, in particular the fact that it is not meant to generate libraries that are optimized for specific metrics.

   > *PoolParty is also not a sequence optimization tool; it does not design sequences that have optimized codon usage (Swainston et al., 2017), that are subject to complex syn-*

thesis constraints (Zulkower and Rosser, 2020), or that have optimal activity as assessed by machine learning models (Schreiber et al., 2025).

6. The name PoolParty may be confused with the existing PoolParty analysis softwre (`https://doi.org/10.1111/1755-0998.12784`).

   This is a fair concern, but we have decided to retain the name PoolParty. The package already uses the name `poolparty` on PyPI and ReadTheDocs, and we have announced this software in a preprint. Moreover, the previously published PoolParty is a pipeline for aligning and analyzing pooled-sequencing data, a use case that we believe is different enough from library sequence generation to avoid confusion with our package.

Reviewer 3

1. I think that the emphasis made on PoolParty being a universal tool that can "replace" (line 256) existing code/tools needs to be qualified more. The benefits of PoolParty are that it is straightforward to use, interpretable, modular, with a nice 'design card' tracking and graphed interpretation – I think this is particularly useful for ML and in silico work.

   However, whist it does streamline design processes, the package operates on input sequences rather than genomic loci co-ordinates/transcript so in terms of wet-lab experiments it is most useful for fundamental biology cDNA DMS and MPRA assays that are episomal (i.e. from plasmid / using reporters) rather than MAVEs that rely upon genome editing (which is becoming the gold standard for clinical variant work). It does not intuitively and natively support reference genome coordinates, transcript models, exon/intron structures, alternative isoforms, or genome assembly awareness (as far as I can tell); VaLiAnT does do this. Base sequences into which variants are installed are represented/input as sequence string (pasted by user) rather than standardized co-ordinate or HGVS notation (this limits interpretation/ingress of variants from clinical and genomics resources). If PoolParty is presented as intuitive and adaptable rather than a universal replacement to alternative tools and code this is less of a concern and may allow researchers to more readily select which software to invest time into learning/using.

   This is a fair critique: PoolParty does not currently operate on genome assemblies together with transcript annotations, nor does it accept HGVS-formatted input. We have removed language suggesting that PoolParty is meant to replace these other tools, including those that do, and have revised the Background and Discussion to define the scope of PoolParty more clearly. In the Supplemental Information, we have added two tables explicitly comparing the features and capabilities of PoolParty with existing tools (Tables S2 and S3), and two figures showing side-by-side implementations of the same libraries in PoolParty versus VaLiAnT or tangermeme (Figs. S3 and S4). Please also see the response to Comment 3 below.

2. Molecular consequence beyond codon change for missense is not reported as far as I can tell, VaLiAnT does this to some extent (e.g. stop-gained, synonymous, inframe deletion, etc, PoolParty may do this also, but I couldn't see this after running a 'replacement_scan'?) but generally researchers use VEP on their tool outputs to achieve extensive annotation. VEP requires

standard annotation or format (e.g. VCF) for input rather than sequence strings – if this is possible for PoolParty (the production of a VCF or a format that VEP will accept) then one could obtain extensive annotations (e.g. SpliceAI predictions, EVE, AlphaMissense, PolyPhen, Consequence, genomic coordinates, Clinvar and gnomAD identifiers etc etc) which I think would greatly increase the uptake of the tool by researchers in the MAVE community that are more clinically/genomically minded – if this is possible I would state this/provide an easy means to generate VEP input format from PoolParty outputs.

Yes, PoolParty does not annotate the molecular consequences of mutations, as it does not return an annotation layer with each sequence. However, if users wish to generate sequences that have a specific type of mutation, then fold these sequences into a larger library, there are two ways to track which specific sequences received which types of mutations: through the design card provided with each sequence, and through each sequence's name.

We appreciate the suggestion to support VCF format. To this end we have implemented a new method, `from_vcf`, which extracts a specified sequence window and inserts the alternative alleles into that sequence. We do not, however, see a simple way of exporting libraries in VCF format, as the sequences generated by PoolParty are often not closely related to any specific wild-type sequence. The best option in this case would be for users to take the sequences generated by PoolParty, align them to a reference, and use the results to generate a corresponding VCF record. We believe that this is best done using other available software, however, as building this capability into PoolParty would substantially expand PoolParty's dependencies.

3. The authors note limitations at the end of the discussion. I think updating this section to address that PoolParty is mostly of benefit for DMS and MPRA rather than genome editing MAVE assays (e.g. not as useful for Saturation Genome Editing HDR libraries and Saturation Prime Editing pegRNA libraries) which require extensive and specific annotation/standardized nomenclature and consideration of transcript (generally through ingress of GTF/GFF files) and exon/intron boundaries (including split codons between exons, again through GTF/GFF boundary annotations).

   Alternatively/additionally the authors might wish to highlight that the tool is agnostic to species/genome/transcript and present it as a more intuitive sequence generation tool which is particularly useful for ML testing/benchmarking which is well tracked with the design cards and interpretable/iteratively mutable through graph interrogation. This builds on the comment that generally annotations are variant specific (line 56-58), with the emphasis being that PoolParty serves as a useful tool to manifest oligo libraries as structured objects (less of a concern for wet-lab investigations in my opinion, but much more salient for ML based studies on raw sequence out of genomic context).

   We agree and have updated the Limitations section accordingly. The revised text reads:

   > *PoolParty operates on sequences, not genomic loci or transcript models. It can accept genomic coordinates as input to extract sequences, but does not interpret transcript annotations or keep track of those coordinates through subsequent design steps (e.g., as VaLiAnT does), which may make PoolParty less helpful for certain genome-editing applications.*

4. Line 12 – there is not a "lack" of purpose-built software, replace with "scarcity" perhaps.

Fair point. We have replaced "lack" with "scarcity." The revised sentence reads:

> *Designing these libraries, however, remains tedious and error-prone due to the scarcity of purpose-built software.*

5. Line 51-53 - stated that VaLinAnT is assay specific, then say used for DMS and saturation genome editing two different assays (SGE is a type of DMS, but VaLinAnT is used for SGE and cDNA DMS which are quite different assays).

   We have replaced "assay-specific" with more general wording and revised the surrounding description. We also tested VaLiAnT on noncoding regions and confirmed that it supports these designs. We have revised the description as follows:

   > *Several other tools automate library design or sequence perturbation for particular applications. For example, VaLiAnT (Barbon et al., 2022) generates mutational scanning libraries of both coding and noncoding regions; supported mutation types include single-nucleotide substitutions, amino acid substitutions, and in-frame deletions.*

6. Line 66 – "expensive computation" this is only true for truly massive sequence generation – most full gene DMS/saturation libraries of >20,000 variants take <5 minutes on a standard laptop CPU (true of Valiant) – this statement would be true for many permutations of sequence for use in ML (e.g. pairwise, full genome CDS on GPU) – I would qualify this point on cost.

   Indeed, most libraries designed for wet-lab experiments can be generated in seconds to minutes on a standard CPU, as the reviewer notes. We have therefore made the wording more specific:

   > *Importantly, no sequences are generated until the user requests them, and it is simple to generate small test sets of sequences. This allows users to explore multiple design options without having to generate complete libraries at each iteration.*

7. The use cases of protein coding, MPRA promoter and splice are good examples (line 68).

   Thanks!

8. Figure 2 B and Figure 3 B – is it necessary helpful to put these code blocks here? I'm not sure how useful this is for readership.

   The purpose of these panels is to support our claim that the PoolParty API is simple to use. We believe that showing the code explicitly, together with the output the code generates, is the best way to support this claim, and so we have opted to retain these panels.

9. Line 194 – 'most abundant codon' – this needs to be defined, is this codon usage (I think not) or instance in the input sequence?

   We have clarified the text:

   > *We note that `mutagenize_orf` supports multiple codon mutation types; this example uses `missense_only_first`, which selects the most frequently used codon in the human genome for each target amino acid.*

10. Line 235 – SpliceAI is defined here for the first time but is introduced before this in section 2.8 – I would move the 'a deep learning model of mRNA splicing' to the earlier section.

    The term SpliceAI is now defined in the earlier Implementation subsection, where this term is first used:

    > *SpliceAI (Jaganathan et al., 2019) is a genomic deep learning model for splice-site prediction in humans. We used the five-model ensemble to predict the canonical 5'ss probability using ±5 kb of genomic context (GRCh38).*

11. Line 238 "varying strength" – maxentscan in legend, I would define this is the main body text too.

    We have clarified that cryptic splice-site strength refers to the MaxEntScan. The revised sentence in Results reads:

    > *Using the 5'ss of SMN2 exon 7 as the canonical site, we used PoolParty to construct a library in which cryptic 5'ss 9-mers of varying strength (MaxEntScan score) were inserted at varying positions on either side of the canonical 5'ss (see Implementation).*

12. Line 248 – Fig.4G is introduced before 4E-F, perhaps generally introduce Fig.4. then this could stay as is.

    We have moved the cross-validation result to the end of the paragraph so that the Figure 4 panels are introduced in order.

13. Line 256 – "replaces the ad hoc" is too bold and a little presumptive as for major points above– needs clarification/qualification. Also, Line 274 – "in contrast"– I think these claims are too strong – valiant can do DMS and MPRA as well as SGE/SPE – I would tone these statements down and/or emphasise what is unique to PoolParty.

    This is fair criticism; we have revised both statements:

    > *We have presented PoolParty, a Python package for designing oligonucleotide libraries. PoolParty facilitates this task by representing libraries as directed acyclic graphs (DAGs) that users can construct using a simple API.*
    >
    > *PoolParty complements existing library-design tools such as VaLiAnT (Barbon et al., 2022) and MPRAnator (Georgakopoulos-Soares et al., 2017) by providing a single framework for DMS libraries, MPRA libraries, and library designs that combine elements of both.*

14. Line 271 – "in the spliceAI example" – the one in section 2.8 or 3.3? (3.3 I imagine, but best to state).

    We have clarified that this refers to the SpliceAI surrogate modeling example in the Results:

    > *For example, in the SpliceAI analysis of Fig. 4, the cryptic splice-site strength and position recorded in each design card provided the covariates used for surrogate modeling.*

15. The code is very well written and documented, and it is noted by the authors that Anthropic models have been used to do this/help which I think is sensible, editorially this may be something to confirm is acceptable at Bioinformatics (as reviewers were requested not to use these tools, but this is likely due to confidentiality).

    BMC Bioinformatics policy permits LLM use when disclosed. We have also clarified the LLM disclosure statement in Implementation:

    > *The authors used large language models (primarily Claude Opus 4.5 and 4.6, Anthropic) to assist with code development, debugging, and documentation writing. The manuscript was written entirely by the authors, with LLM assistance only at the copy-editing stage. The figures were also prepared by the authors; LLMs were used only to help write the scripts that generate the plots. The authors affirm that they are fully responsible for the content of the manuscript, the codebase, and the documentation.*

16. The name 'PoolParty' is great.

    Thank you!

References

Cock PJA, Antao T, Chang JT, Chapman BA, Cox CJ, Dalke A, et al. Biopython: freely available Python tools for computational molecular biology and bioinformatics. Bioinformatics. 2009;25(11):1422–1423. https://doi.org/10.1093/bioinformatics/btp163.

Pereira F, Azevedo F, Carvalho Â, Ribeiro GF, Budde MW, Johansson B. Pydna: a simulation and documentation tool for DNA assembly strategies using python. BMC Bioinformatics. 2015;16(1):142. https://doi.org/10.1186/s12859-015-0544-x.

Klie A, Laub D. SeqPro (Sequence processing toolkit); 2023. Software. Available from: https://github.com/ML4GLand/SeqPro.

Barbon L, Offord V, Radford EJ, Butler AP, Gerety SS, Adams DJ, et al. Variant Library Annotation Tool (VaLiAnT): an oligonucleotide library design and annotation tool for saturation genome editing and other deep mutational scanning experiments. Bioinformatics. 2022;38(4):892–899. https://doi.org/10.1093/bioinformatics/btab776.

Georgakopoulos-Soares I, Jain N, Gray JM, Hemberg M. MPRAnator: a web-based tool for the design of massively parallel reporter assay experiments. Bioinformatics. 2017;33(1):137–138. https://doi.org/10.1093/bioinformatics/btw584.

Ghazi AR, Chen ES, Henke DM, Madan N, Edelstein LC, Shaw CA. Design tools for MPRA experiments. Bioinformatics. 2018;34(15):2682–2683. https://doi.org/10.1093/bioinformatics/bty150.

Schreiber J. tangermeme: A toolkit for understanding cis-regulatory logic using deep learning models. bioRxiv. 2025;p. 2025.08.08.669296. https://doi.org/10.1101/2025.08.08.669296.

Swainston N, Currin A, Green L, Breitling R, Day PJ, Kell DB. CodonGenie: optimised ambiguous codon design tools. PeerJ Comput Sci. 2017;3:e120. https://doi.org/10.7717/peerj-cs.120.

Zulkower V, Rosser S. DNA Chisel, a versatile sequence optimizer. Bioinformatics. 2020;36(16):4508–4509. https://doi.org/10.1093/bioinformatics/btaa558.

Schreiber J, Lorbeer FK, Heinzl M, Lu YY, Stark A, Noble WS. Programmatic design and editing of cis-regulatory elements. bioRxiv. 2025;p. 2025.04.22.650035. https://doi.org/10.1101/2025.04.22.650035.

Hiraga K, Mejzlik P, Marcisin M, Vostrosablin N, Gromek A, Arnold J, et al. Mutation Maker, An Open Source Oligo Design Platform for Protein Engineering. ACS Synth Biol. 2021;10(2):357–370. https://doi.org/10.1021/acssynbio.0c00542.

Jaganathan K, Panagiotopoulou SK, McRae JF, Darbandi SF, Knowles D, Li YI, et al. Predicting Splicing from Primary Sequence with Deep Learning. Cell. 2019 Jan;176(3):535–548.e24. https://doi.org/10.1016/j.cell.2018.12.015.